ASSIGNMENT 6

Mansha Bhatnagar
1024150192
2O23

Q1 AND 2

Topics:

1. Array of objects, Constructors in array of objects
2. Pointer to objects, this pointer
3. Passing and returning objects
4. Static and const member function
5. Inline function

1. Implement a class ***Library*** that contains **an array of *Book objects***. The ***Book*** class should have attributes like **title, author, and ISBN**. The ***Library*** class should provide following methods:

   a. **bool addNewBook(title,author,ISBN)** to add the new book in the book list. The return type should be Boolean to check whether the book is added or not. This function should take the argument using pass-by-reference method. The attribute of the class (title, author, and ISBN) and the function arguments of function **addNewBook** should be same. Use the **scope resolution operator** to access the class data members.

   b. **bool removeBooks(ISBN)** to remove the book using the input parameter ISBN. The return type should be Boolean to check whether the book is removed or not. This function also should take the argument using pass-by-reference method. Define function outside the class using scope resolution operator.

   c. **displayDetails()** to list the books.

   Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

2. Implement the question 1 using constructor. Do the following modification:

   a. Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., **title, author, and ISBN.**

b. Use the "this" pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.

c. Create function **bool removeBooks(ISBN)** same as the question 1.

d. **displayDetails()** same as question 1.

e. Create array of objects using 1. Initializer list, 2. Dynamic initialization

```cpp
#include <iostream>
#include <string>
using namespace std;

class Book {
public:
    string title, author, ISBN;

    Book() {}
    Book(string title, string author, string ISBN) {
        this->title = title;
        this->author = author;
        this->ISBN = ISBN;
    }
    Book(const Book &b) {
        this->title = b.title;
        this->author = b.author;
        this->ISBN = b.ISBN;
    }
};

class Library {
    Book books[10];
    int count = 0;
public:
    bool addNewBook(const string &title, const string &author, const str
    bool removeBooks(const string &ISBN);
    void displayDetails();
};

bool Library::addNewBook(const string &title, const string &author, cons
    if (count < 10) {
        books[count] = Book(title, author, ISBN);
        count++;
        return true;
    }
    return false;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local\Temp\" ; if ($?) {
++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\
mpCodeRunnerFile }
Library after adding 5 books:
Title: The Alchemist | Author: Paulo Coelho | ISBN: 101
Title: 1984 | Author: George Orwell | ISBN: 102
Title: Hamlet | Author: Shakespeare | ISBN: 103
Title: Deep Work | Author: Cal Newport | ISBN: 104
Title: Atomic Habits | Author: James Clear | ISBN: 105

After removing ISBN 103:
Title: The Alchemist | Author: Paulo Coelho | ISBN: 101
Title: 1984 | Author: George Orwell | ISBN: 102
Title: Deep Work | Author: Cal Newport | ISBN: 104
Title: Atomic Habits | Author: James Clear | ISBN: 105
PS C:\Users\HP\AppData\Local\Temp>
```

```cpp
bool Library::removeBooks(const string &ISBN) {
    for (int i = 0; i < count; i++) {
        if (this->books[i].ISBN == ISBN) {
            for (int j = i; j < count - 1; j++) {
                books[j] = books[j + 1];
            }
            count--;
            return true;
        }
    }
    return false;
}

void Library::displayDetails() {
    for (int i = 0; i < count; i++) {
        cout << "Title: " << books[i].title << " | Author: " << books[i]
    }
}

int main() {
    Library lib;
    lib.addNewBook("The Alchemist", "Paulo Coelho", "101");
    lib.addNewBook("1984", "George Orwell", "102");
    lib.addNewBook("Hamlet", "Shakespeare", "103");
    lib.addNewBook("Deep Work", "Cal Newport", "104");
    lib.addNewBook("Atomic Habits", "James Clear", "105");

    cout << "Library after adding 5 books:" << endl;
    lib.displayDetails();

    lib.removeBooks("103");
    cout << "\nAfter removing ISBN 103:" << endl;
    lib.displayDetails();

    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local\Temp\" ; if ($?) { g
++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\te
mpCodeRunnerFile }
Library after adding 5 books:
Title: The Alchemist | Author: Paulo Coelho | ISBN: 101
Title: 1984 | Author: George Orwell | ISBN: 102
Title: Hamlet | Author: Shakespeare | ISBN: 103
Title: Deep Work | Author: Cal Newport | ISBN: 104
Title: Atomic Habits | Author: James Clear | ISBN: 105

After removing ISBN 103:
Title: The Alchemist | Author: Paulo Coelho | ISBN: 101
Title: 1984 | Author: George Orwell | ISBN: 102
Title: Deep Work | Author: Cal Newport | ISBN: 104
Title: Atomic Habits | Author: James Clear | ISBN: 105
PS C:\Users\HP\AppData\Local\Temp>
```

Q3

e. Create array of objects using 1. Initializer list, 2. Dynamic initialization

Create a class ***Account*** with attributes ***account number (of const long type), transaction ID (of long type), transaction type (credit/debit) (of string type), and balance (of double type).*** The Account class should provide following methods:

a. **long depositAmount(to, from, amount)**. The "to" and "from" are the account number of the sender and receiver. The function should return the transaction id.

b. **long creditAmount(to, from, amount)** similar to the depositAmount function. This function should also return the transaction id.

c. **void displayDetails()** should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.

d. In all the functions, make all the arguments as the const arguments.

Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit.

```cpp
#include <iostream>
#include <string>

class Account {
    const long accNum;
    long transID;
    std::string transType;
    double balance;

public:
    Account(long n, double b) : accNum(n), balance(b) {
        transID = 0;
        transType = "None";
    }

    long depositAmount(const long &to, const long &from, const double &
        balance += amount;
        transType = "Credit";
        transID = 1000 + (accNum % 100);
        return transID;
    }

    long creditAmount(const long &to, const long &from, const double &a
        if (balance >= amount) {
            balance -= amount;
            transType = "Debit";
            transID = 2000 + (accNum % 100);
            return transID;
        }
        return -1;
    }

    void displayDetails() const {
        std::cout << "Acc: " << accNum << " | Balance: " << bala
                  << " | Last Trans: " << transType << " | ID: " << tra
    }
};
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local\Temp\" ; if ($
++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) {
mpCodeRunnerFile }
Acc: 101 | Balance: 5500 | Last Trans: Credit | ID: 1001
Acc: 102 | Balance: 2800 | Last Trans: Debit | ID: 2002
Acc: 103 | Balance: 7000 | Last Trans: None | ID: 0
Acc: 104 | Balance: 2000 | Last Trans: None | ID: 0
Acc: 105 | Balance: 10000 | Last Trans: None | ID: 0
PS C:\Users\HP\AppData\Local\Temp>
```

```cpp
public:
    void displayDetails() const {
        std::cout << "Acc: " << accNum << " | Balance: " << bala
                  << " | Last Trans: " << transType << " | ID: "
    }
};

int main() {
    Account accs[5] = {
        Account(101, 5000), Account(102, 3000),
        Account(103, 7000), Account(104, 2000),
        Account(105, 10000)
    };

    accs[0].depositAmount(101, 999, 500);
    accs[1].creditAmount(999, 102, 200);

    for(int i = 0; i < 5; i++) {
        accs[i].displayDetails();
    }

    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local\Temp\" ; if
++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?
mpCodeRunnerFile }
Acc: 101 | Balance: 5500 | Last Trans: Credit | ID: 1001
Acc: 102 | Balance: 2800 | Last Trans: Debit | ID: 2002
Acc: 103 | Balance: 7000 | Last Trans: None | ID: 0
Acc: 104 | Balance: 2000 | Last Trans: None | ID: 0
Acc: 105 | Balance: 10000 | Last Trans: None | ID: 0
PS C:\Users\HP\AppData\Local\Temp>
```

Q4 A. Write a program to add data objects of two different classes using friend functions.

```cpp
#include <iostream>
using namespace std;

class ClassB;

class ClassA {
    int valA;
public:
    ClassA(int v) : valA(v) {}
    friend int add(ClassA, ClassB);
};

class ClassB {
    int valB;
public:
    ClassB(int v) : valB(v) {}
    friend int add(ClassA, ClassB);
};

int add(ClassA objA, ClassB objB) {
    return objA.valA + objB.valB;
}

int main() {
    ClassA a(10);
    ClassB b(20);
    cout << "Sum using friend function: " << add(a, b) << endl;
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppDat
++ tempCodeRunnerFile.cpp -o tempCodeR
mpCodeRunnerFile }
Sum using friend function: 30
PS C:\Users\HP\AppData\Local\Temp>
```

5. Define a class named ***Complex*** with properties (real and imaginary) and methods as per following details.

   a. ***Parameterized constructor*** and ***copy constructor*** to initialize object values.

   b. ***void display ()*** to display complex number.

   c. ***void sum (Complex, Complex)*** to add two complex numbers (objects of Complex class). Implement this function as the friend function.

==========*******==========

```cpp
#include <iostream>
using namespace std;

class Complex {
    float real, imag;
public:
    Complex() : real(0), imag(0) {}
    Complex(float r, float i) : real(r), imag(i) {}
    Complex(const Complex &c) {
        real = c.real;
        imag = c.imag;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }

    friend Complex sum(Complex, Complex);
};

Complex sum(Complex c1, Complex c2) {
    Complex temp;
    temp.real = c1.real + c2.real;
    temp.imag = c1.imag + c2.imag;
    return temp;
}

int main() {
    Complex c1(3.5, 2.5), c2(1.5, 4.5);
    Complex res = sum(c1, c2);

    cout << "C1: "; c1.display();
    cout << "C2: "; c2.display();
    cout << "Sum: "; res.display();

    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\
deRunnerFile.cpp -o tempCodeRunnerFile }
C1: 3.5 + 2.5i
C2: 1.5 + 4.5i
Sum: 5 + 7i
PS C:\Users\HP\AppData\Local\Temp>
```